

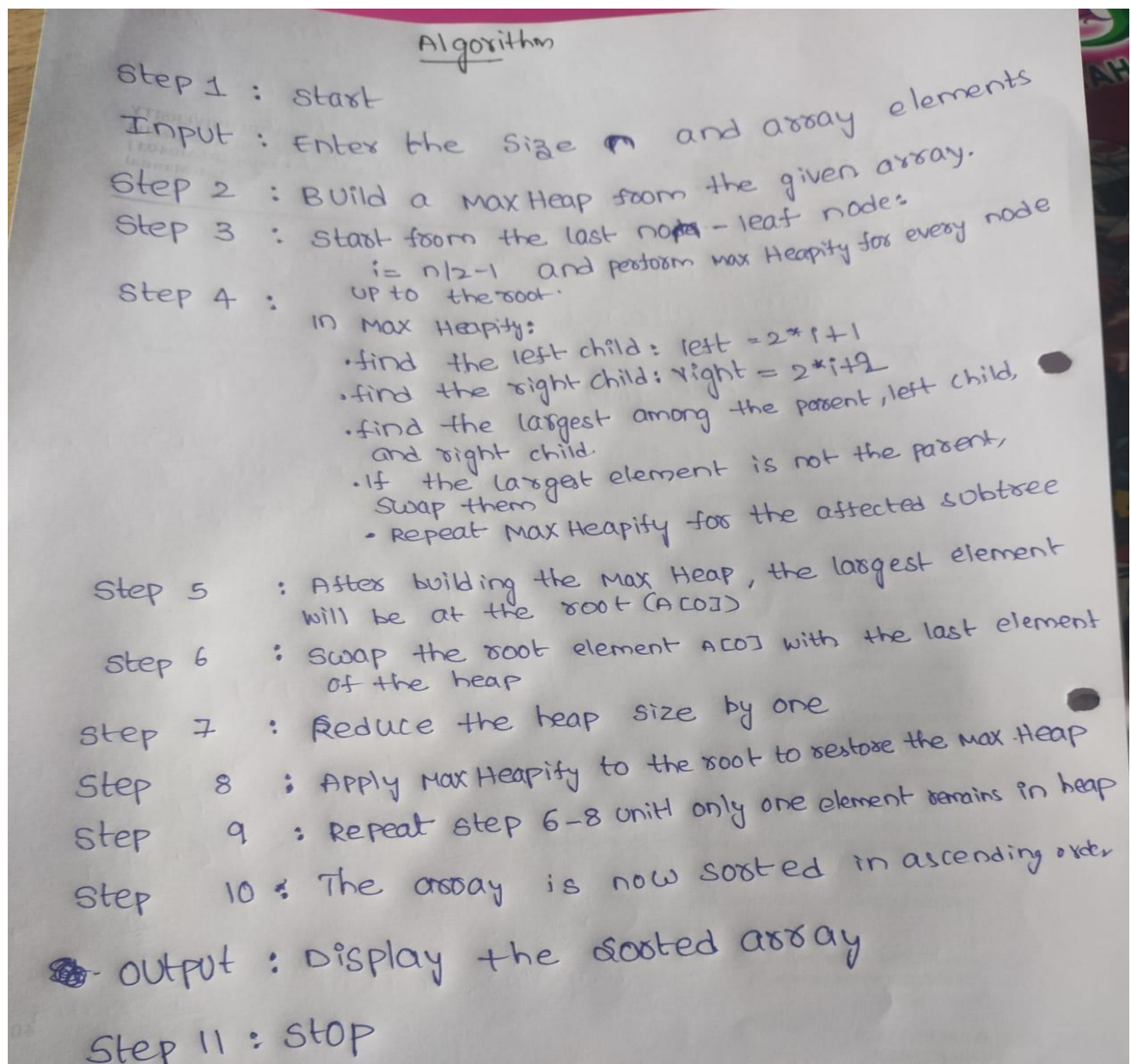
Practical: 3

Implementation of max-heap sort algorithm

/08/2026

Aim: Implementation of max-heap sort algorithm

Algorithm:



The image shows a handwritten document titled "Algorithm" detailing the steps for Max-Heap Sort. The steps are as follows:

- Step 1 : start
- Input : Enter the size n and array elements
- Step 2 : Build a MaxHeap from the given array.
- Step 3 : Start from the last node - leaf nodes
 $i = n/2 - 1$ and perform max Heapify for every node up to the root.
- Step 4 :
 - in Max Heapify:
 - find the left child: $left = 2*i + 1$
 - find the right child: $right = 2*i + 2$
 - find the largest among the parent, left child, and right child.
 - If the largest element is not the parent, Swap them
 - Repeat Max Heapify for the affected subtree
- Step 5 : After building the Max Heap, the largest element will be at the root ($A[0]$)
- Step 6 : Swap the root element $A[0]$ with the last element of the heap
- Step 7 : Reduce the heap size by one
- Step 8 : Apply Max Heapify to the root to restore the Max Heap
- Step 9 : Repeat step 6-8 until only one element remains in heap
- Step 10 : The array is now sorted in ascending order
- Output : Display the sorted array
- Step 11 : stop

Program:

```
#include <iostream>
```

```
using namespace std;

class HeapSort
{
public:

    void maxHeapify(int arr[], int size, int i)
    {
        int length = i;

        int l = (2 * i);
        int r = (2 * i) + 1;

        while (l <= size && arr[l] > arr[length])
        {
            length = l;
        }

        while (r <= size && arr[r] > arr[length])
        {
            length = r;
        }

        if (length != i)
        {
            swap(arr[i], arr[length]);

            maxHeapify(arr, size, length);
        }
    }
}
```

```
    }  
}  
  
void heapSort(int arr[], int size)  
{  
    for (int i = size / 2; i >= 1; i--)  
    {  
        maxHeapify(arr, size, i);  
    }  
  
    for (int i = size; i >= 2; i--)  
    {  
        swap(arr[1], arr[i]);  
  
        maxHeapify(arr, size-1, 1);  
    }  
}  
  
void display(int arr[], int size)  
{  
    cout << "Sorted Array: ";  
  
    for (int i = 1; i <= size; i++)  
    {  
        cout << arr[i] << " ";  
    }  
  
    cout << endl;
```

```
    }  
};  
  
int main()  
{  
    HeapSort h;  
  
    int arr[100];  
    int size;  
  
    cout << "Enter size of array: ";  
    cin >> size;  
  
    cout << "Enter array elements: ";  
  
    for (int i = 1; i <= size; i++)  
    {  
        cin >> arr[i];  
    }  
  
    h.heapSort(arr, size);  
  
    h.display(arr, size);  
  
    return 0;  
}
```

Output:

```
C:\Users\Admin\Documents\l × + v
Enter size of array:
7
Enter array elements: 9
10
0
20
50
100
45
Sorted Array: 50 45 20 9 10 0 100

-----
Process exited after 160.2 seconds with return value 0
Press any key to continue . . . |
```

Time Complexity:

Time complexity

```

MAX Heapify (arr, size, i) {  $\rightarrow T(n)$ 
    int largest = i;  $\rightarrow O(1)$ 
    int l = (2*i);  $\rightarrow O(1)$ 
    int r = (2*i)+1;  $\rightarrow O(1)$ 
    while (l <= size && arr[l] > arr[largest]) {
        largest = l;  $\rightarrow O(1)$ 
    }
    while (r <= size && arr[r] > arr[largest]) {
        largest = r;  $\rightarrow O(1)$ 
    }
    if (largest != i)  $\rightarrow O(1)$ 
    {
        swap (arr[i], arr[largest]);  $\rightarrow O(1)$ 
        MAX Heapify (arr, size, largest);  $\rightarrow O(1)$ 
    }
}

void heapsort (int arr[], int size)  $\rightarrow O(1)$ 
{
    for (int i = size/2; i >= 1; i--)  $\rightarrow T(n/2)$ 
    {
        max Heapify (arr, size, i);  $\rightarrow O(1)$ 
    }
    for (int i = size; i >= 2; i--)  $\rightarrow O(1)$ 
    {
        swap (arr[1], arr[i]);  $\rightarrow O(1)$ 
        Max Heapify (arr, size-1, 1);  $\rightarrow O(1)$ 
    }
}
    
```



Worst case

the worst case, $\text{maxheapify}()$ can travel down the height of the heap.

for each extraction: $O(\log n)$

there are approximately n extractions:

$$T(n) = n \log n$$

Build Heap: $O(n)$

Heap sort:

$$n \times O(\log n)$$

$$= O(n \log n)$$

$$\text{total} : O(n) + O(n \log n)$$

$$= O(n \log n)$$

$$\text{Space} = O(\log n)$$

therefore:

$$\text{worst case} = O(n \log n)$$

TIME Complexity

Best case

even if the array sorted, your heap sort() still performs:

```
for (int i = size; i > 2; i--)
```

and calls maxHeapify() after every execution

Therefore, the sorting phase still takes: $O(n \log n)$

so

Best case = $O(n \log n)$

Average case

for an Average/Random input, the heap is built and elements are repeatedly extracted

Build heap: $O(n)$

extraction: $(n-1) O(\log n)$

Therefore

$$T(n) = O(n) + (n-1) O(\log n)$$

$$= O(n \log n)$$

so

Average case = $O(n \log n)$

Conclusion:

Max-Heap Sort was successfully implemented to sort the given elements in ascending order. The algorithm works by building a max heap and repeatedly placing the largest element in its correct position.